System Design

Code review and analysis

Vincent Tran

Recap: Homework 3

Complete the `ApiHandler` class in `src/handler/api.ts`, and submit:

- all source code, and
- a report (PDF) with a screenshot of the service's performance statistics and a short analysis.

Engineering currency

Engineering currency

1. **Machine currency, the "real" cost.** CPU cycles, memory, database I/O, network calls. In industry, efficiency is money.
2. **Human currency, the "cognitive" cost.** Readability, maintainability, time to understand.
   - Code is read far more often than it is written.
   - Cognitive load: how much effort does it take to see what the code does?
   - Debug time: how fast can you fix it?

Machine currency

In the homework, efficiency is measured in units. In the real world, units become a computing bill.

An illustration. Suppose 1 unit = \$0.0001, over 10k requests:

- Implementation X costs 1.5 million units: \$150.00
- Implementation Y costs 140k units: \$14.00

Machine currency

In the homework, efficiency is measured in units. In the real world, units become a computing bill.

An illustration. Suppose 1 unit = \$0.0001, over 10k requests:

- Implementation X costs 1.5 million units: \$150.00
- Implementation Y costs 140k units: \$14.00

Scale to 1 million users (100× the requests), against a \$10k monthly budget:

- X's bill: \$15k a month. Over budget: bankruptcy.
- Y's bill: \$1.4k a month. Well under: profit!

What the simulation charges

Every request in the HW3 simulation also pays:

- 128 units to check the token with the Identity Provider
- 256 units for *every* `getInstance()` call on the database table or an external client

So your totals will be well above the illustration. Compare them with each other.

Human currency

Key metrics:

- Time to understand (TTU)
- Mean time to repair (MTTR)

Higher TTU means higher MTTR.

Human currency

Key metrics:

- Time to understand (TTU)
- Mean time to repair (MTTR)

Higher TTU means higher MTTR.

It's 3:00 AM and the production server is down.

- **Complex code:** two hours or more to fix, lost revenue, escalation to your manager.
- **Clean code:** under ten minutes to fix; customers may never notice.

Code samples

Re-reading what you just wrote

A

```
private handleRequestMachine(
    request: RequestMachineRequestModel,
): MachineResponseModel {
    const table = MachineStateTable.getInstance();
    const machines = table.listMachinesAtLocation(request.locationId);
    const available = machines.find(m => m.status === MachineStatus.AVAILABLE);
    if (!available) {
        return { statusCode: HttpResponseCode.NOT_FOUND, machine: undefined };
    }
    table.updateMachineJobId(available.machineId, request.jobId);
    table.updateMachineStatus(
        available.machineId, MachineStatus.AWAITING_DROPOFF);
    const updated = table.getMachine(available.machineId)!;
    this.cache.put(updated.machineId, updated);
    return { statusCode: HttpResponseCode.OK, machine: updated };
}
```

Re-reading what you just wrote

A

```
private handleRequestMachine(
    request: RequestMachineRequestModel,
): MachineResponseModel {
    const table = MachineStateTable.getInstance();
    const machines = table.listMachinesAtLocation(request.locationId);
    const available = machines.find(m => m.status === MachineStatus.AVAILABLE);
    if (!available) {
        return { statusCode: HttpResponseCode.NOT_FOUND, machine: undefined };
    }
    table.updateMachineJobId(available.machineId, request.jobId);
    table.updateMachineStatus(
        available.machineId, MachineStatus.AWAITING_DROPOFF);
    const updated = table.getMachine(available.machineId)!;
    this.cache.put(updated.machineId, updated);
    return { statusCode: HttpResponseCode.OK, machine: updated };
}
```

- Updates the database, then immediately queries it.
- An unnecessary read.

Re-reading what you just wrote

A

```
private handleRequestMachine(
    request: RequestMachineRequestModel,
): MachineResponseModel {
    const table = MachineStateTable.getInstance();
    const machines = table.listMachinesAtLocation(request.locationId);
    const available = machines.find(m => m.status === MachineStatus.AVAILABLE);
    if (!available) {
        return { statusCode: HttpResponseCode.NOT_FOUND, machine: undefined };
    }
    table.updateMachineJobId(available.machineId, request.jobId);
    table.updateMachineStatus(
        available.machineId, MachineStatus.AWAITING_DROPOFF);
    const updated = table.getMachine(available.machineId)!;
    this.cache.put(updated.machineId, updated);
    return { statusCode: HttpResponseCode.OK, machine: updated };
}
```

- Updates the database, then immediately queries it.
- An unnecessary read.
- **Why?** Lack of confidence in local state: the programmer wants to be “sure” they have the latest data.

Building the result in memory

B

```
private handleRequestMachine(
    request: RequestMachineRequestModel,
): MachineResponseModel {
    const table = MachineStateTable.getInstance();
    const machines = table.listMachinesAtLocation(request.locationId);
    var available = machines.find(m => m.status === MachineStatus.AVAILABLE);
    if (!available) {
        return { statusCode: HttpResponseCode.NOT_FOUND, machine: undefined };
    }
    const updatedMachine = {
        machineId: available.machineId,
        locationId: available.locationId,
        currentJobId: request.jobId,
        status: MachineStatus.AWAITING_DROPOFF
    }
    table.updateMachineStatus(updatedMachine.machineId, updatedMachine.status);
    table.updateMachineJobId(
        updatedMachine.machineId, updatedMachine.currentJobId);
    this.cache.put(updatedMachine.machineId, updatedMachine);
    return { statusCode: HttpResponseCode.OK, machine: updatedMachine };
}
```

Building the result in memory

B

```
private handleRequestMachine(
    request: RequestMachineRequestModel,
): MachineResponseModel {
    const table = MachineStateTable.getInstance();
    const machines = table.listMachinesAtLocation(request.locationId);
    var available = machines.find(m => m.status === MachineStatus.AVAILABLE);
    if (!available) {
        return { statusCode: HttpResponseCode.NOT_FOUND, machine: undefined };
    }
    const updatedMachine = {
        machineId: available.machineId,
        locationId: available.locationId,
        currentJobId: request.jobId,
        status: MachineStatus.AWAITING_DROPOFF
    }
    table.updateMachineStatus(updatedMachine.machineId, updatedMachine.status);
    table.updateMachineJobId(
        updatedMachine.machineId, updatedMachine.currentJobId);
    this.cache.put(updatedMachine.machineId, updatedMachine);
    return { statusCode: HttpResponseCode.OK, machine: updatedMachine };
}
```

- Constructs the updated item in memory.
- That costs nothing!

Building the result in memory

B

```
private handleRequestMachine(
    request: RequestMachineRequestModel,
): MachineResponseModel {
    const table = MachineStateTable.getInstance();
    const machines = table.listMachinesAtLocation(request.locationId);
    var available = machines.find(m => m.status === MachineStatus.AVAILABLE);
    if (!available) {
        return { statusCode: HttpResponseCode.NOT_FOUND, machine: undefined };
    }
    const updatedMachine = {
        machineId: available.machineId,
        locationId: available.locationId,
        currentJobId: request.jobId,
        status: MachineStatus.AWAITING_DROPOFF
    }
    table.updateMachineStatus(updatedMachine.machineId, updatedMachine.status);
    table.updateMachineJobId(
        updatedMachine.machineId, updatedMachine.currentJobId);
    this.cache.put(updatedMachine.machineId, updatedMachine);
    return { statusCode: HttpResponseCode.OK, machine: updatedMachine };
}
```

- Constructs the updated item in memory.
- That costs nothing!
- **Takeaway:** if you own the data transformation, don't pay the database to repeat back what you just did.

Building the result in memory

B

```
private handleRequestMachine(
    request: RequestMachineRequestModel,
): MachineResponseModel {
    const table = MachineStateTable.getInstance();
    const machines = table.listMachinesAtLocation(request.locationId);
    var available = machines.find(m => m.status === MachineStatus.AVAILABLE);
    if (!available) {
        return { statusCode: HttpResponseCode.NOT_FOUND, machine: undefined };
    }
    const updatedMachine = {
        machineId: available.machineId,
        locationId: available.locationId,
        currentJobId: request.jobId,
        status: MachineStatus.AWAITING_DROPOFF
    }
    table.updateMachineStatus(updatedMachine.machineId, updatedMachine.status);
    table.updateMachineJobId(
        updatedMachine.machineId, updatedMachine.currentJobId);
    this.cache.put(updatedMachine.machineId, updatedMachine);
    return { statusCode: HttpResponseCode.OK, machine: updatedMachine };
}
```

- Constructs the updated item in memory.
- That costs nothing!
- **Takeaway:** if you own the data transformation, don't pay the database to repeat back what you just did.
- **Aside:** `var` makes readers check for reassignment. Prefer `const`.

A versus B

Operation	A (re-read)	B (in memory)
List machines	8	8
Update status	15	15
Update job	12	12
Get result	8 (database read)	0 (memory)
Cache put	6	6
Total units	**49**	**41**

Database and cache work only; both also pay the same connection and token charges.
Re-reading adds 8 units: about 20% more than B.

Two requests, one machine

A and B both read the list, pick an available machine, then write to it.

Two requests, one machine

A and B both read the list, pick an available machine, then write to it.

- Two requests for the same location arrive together.
- Both pick the same machine. Both reserve it.

Two requests, one machine

A and B both read the list, pick an available machine, then write to it.

- Two requests for the same location arrive together.
- Both pick the same machine. Both reserve it.

How would you prevent that? A conditional write that succeeds only if the machine is still `AVAILABLE` (compare-and-set), a lock, or a transaction.

Cache lookaside

C

```
private handleGetMachine(
    request: GetMachineRequestModel,
): MachineResponseModel {
    const table = MachineStateTable.getInstance();
    const cached = this.cache.get(request.machineId);
    if (cached) return { statusCode: HttpResponseCode.OK, machine: cached };
    const doc = table.getMachine(request.machineId);
    if (!doc) return { statusCode: HttpResponseCode.NOT_FOUND };
    this.cache.put(request.machineId, doc);
    return { statusCode: HttpResponseCode.OK, machine: doc };
}
```

Cache lookaside

C

```
private handleGetMachine(
    request: GetMachineRequestModel,
): MachineResponseModel {
    const table = MachineStateTable.getInstance();
    const cached = this.cache.get(request.machineId);
    if (cached) return { statusCode: HttpResponseCode.OK, machine: cached };
    const doc = table.getMachine(request.machineId);
    if (!doc) return { statusCode: HttpResponseCode.NOT_FOUND };
    this.cache.put(request.machineId, doc);
    return { statusCode: HttpResponseCode.OK, machine: doc };
}
```

- This is good.

Cache lookaside

C

```
private handleGetMachine(
    request: GetMachineRequestModel,
): MachineResponseModel {
    const table = MachineStateTable.getInstance();
    const cached = this.cache.get(request.machineId);
    if (cached) return { statusCode: HttpResponseCode.OK, machine: cached };
    const doc = table.getMachine(request.machineId);
    if (!doc) return { statusCode: HttpResponseCode.NOT_FOUND };
    this.cache.put(request.machineId, doc);
    return { statusCode: HttpResponseCode.OK, machine: doc };
}
```

- This is good.
- **Cache lookaside:** checks the cache first and skips the database on a hit.

Cache lookaside

C

```
private handleGetMachine(
    request: GetMachineRequestModel,
): MachineResponseModel {
    const table = MachineStateTable.getInstance();
    const cached = this.cache.get(request.machineId);
    if (cached) return { statusCode: HttpResponseCode.OK, machine: cached };
    const doc = table.getMachine(request.machineId);
    if (!doc) return { statusCode: HttpResponseCode.NOT_FOUND };
    this.cache.put(request.machineId, doc);
    return { statusCode: HttpResponseCode.OK, machine: doc };
}
```

- This is good.
- **Cache lookaside:** checks the cache first and skips the database on a hit.
- **Return early:** readable, no deep nesting.

Cache lookaside

C

```
private handleGetMachine(
    request: GetMachineRequestModel,
): MachineResponseModel {
    const table = MachineStateTable.getInstance();
    const cached = this.cache.get(request.machineId);
    if (cached) return { statusCode: HttpResponseCode.OK, machine: cached };
    const doc = table.getMachine(request.machineId);
    if (!doc) return { statusCode: HttpResponseCode.NOT_FOUND };
    this.cache.put(request.machineId, doc);
    return { statusCode: HttpResponseCode.OK, machine: doc };
}
```

- This is good.
- **Cache lookaside:** checks the cache first and skips the database on a hit.
- **Return early:** readable, no deep nesting.
- **But:** `getInstance()` runs before the cache check, so even a hit pays a 256-unit connection. Get the table only on a miss, or once in the constructor.

Is the cache worth it?

Counting only cache and database operations:

Path	Units per call
Cache hit	0
Cache miss: miss 4 + database read 8 + cache write 6	18
No cache: database read	8

Is the cache worth it?

Counting only cache and database operations:

Path	Units per call
Cache hit	0
Cache miss: miss 4 + database read 8 + cache write 6	18
No cache: database read	8

Over 10k calls at a 50% hit rate: 90k units with the cache, 80k without.

Is the cache worth it?

Counting only cache and database operations:

Path	Units per call
Cache hit	0
Cache miss: miss 4 + database read 8 + cache write 6	18
No cache: database read	8

Over 10k calls at a 50% hit rate: 90k units with the cache, 80k without.
Caching pays off only above about a 56% hit rate. What hit rate did *your* simulation run report?

Extra work in the error path

D

```
private handleStartMachine(
    request: StartMachineRequestModel,
): MachineResponseModel {
    // ... [Validation logic omitted] ...
    const smartMachine = SmartMachineClient.getInstance();
    try {
        smartMachine.startCycle(request.machineId);
        // ... [Success logic omitted] ...
    }
    catch (err) {
        db.updateMachineStatus(request.machineId, MachineStatus.ERROR);
        machine = db.getMachine(request.machineId);
        if (machine) {
            this.cache.put(request.machineId, machine);
        }
        return { statusCode: HttpResponseCode.HARDWARE_ERROR, machine };
    }
}
```

Extra work in the error path

D

```
private handleStartMachine(
    request: StartMachineRequestModel,
): MachineResponseModel {
    // ... [Validation logic omitted] ...
    const smartMachine = SmartMachineClient.getInstance();
    try {
        smartMachine.startCycle(request.machineId);
        // ... [Success logic omitted] ...
    }
    catch (err) {
        db.updateMachineStatus(request.machineId, MachineStatus.ERROR);
        machine = db.getMachine(request.machineId);
        if (machine) {
            this.cache.put(request.machineId, machine);
        }
        return { statusCode: HttpResponseCode.HARDWARE_ERROR, machine };
    }
}
```

- On a hardware failure, control jumps to the `catch`.
- Writing `ERROR` is necessary; re-reading the machine is Sample A's extra read, on a path that is already failing.

Extra work in the error path

D

```
private handleStartMachine(
    request: StartMachineRequestModel,
): MachineResponseModel {
    // ... [Validation logic omitted] ...
    const smartMachine = SmartMachineClient.getInstance();
    try {
        smartMachine.startCycle(request.machineId);
        // ... [Success logic omitted] ...
    }
    catch (err) {
        db.updateMachineStatus(request.machineId, MachineStatus.ERROR);
        machine = db.getMachine(request.machineId);
        if (machine) {
            this.cache.put(request.machineId, machine);
        }
        return { statusCode: HttpResponseCode.HARDWARE_ERROR, machine };
    }
}
```

- On a hardware failure, control jumps to the `catch`.
- Writing `ERROR` is necessary; re-reading the machine is Sample A's extra read, on a path that is already failing.
- Bigger questions: is the machine left half-started? Should it call `forceStop`, retry, or time out?

Idiomatic code that hides a cost

E

```
private handleRequestMachine(
    request: RequestMachineRequestModel,
): MachineResponseModel {
    const db = MachineStateTable.getInstance();
    const availableMachines = db.listMachinesAtLocation(request.locationId)
        .filter(m => m.status === MachineStatus.AVAILABLE);
    if (availableMachines.length === 0) {
        return { statusCode: HttpResponseCode.NOT_FOUND };
    }
    const machine = availableMachines[0];
    // ... [Update logic omitted] ...
}
```

Idiomatic code that hides a cost

```
private handleRequestMachine(
    request: RequestMachineRequestModel,
): MachineResponseModel {
    const db = MachineStateTable.getInstance();
    const availableMachines = db.listMachinesAtLocation(request.locationId)
        .filter(m => m.status === MachineStatus.AVAILABLE);
    if (availableMachines.length === 0) {
        return { statusCode: HttpResponseCode.NOT_FOUND };
    }
    const machine = availableMachines[0];
    // ... [Update logic omitted] ...
}
```

E

- What does `.filter(...)` do? Check the documentation (or ask an AI).

Idiomatic code that hides a cost

```
private handleRequestMachine(
    request: RequestMachineRequestModel,
): MachineResponseModel {
    const db = MachineStateTable.getInstance();
    const availableMachines = db.listMachinesAtLocation(request.locationId)
        .filter(m => m.status === MachineStatus.AVAILABLE);
    if (availableMachines.length === 0) {
        return { statusCode: HttpResponseCode.NOT_FOUND };
    }
    const machine = availableMachines[0];
    // ... [Update logic omitted] ...
}
```

E

- What does `.filter(...)` do? Check the documentation (or ask an AI).
- It looks clean but hides an efficiency flaw: it walks the *entire* array. A location with 1,000 machines checks all of them, every time.

Idiomatic code that hides a cost

```
private handleRequestMachine(
    request: RequestMachineRequestModel,
): MachineResponseModel {
    const db = MachineStateTable.getInstance();
    const availableMachines = db.listMachinesAtLocation(request.locationId)
        .filter(m => m.status === MachineStatus.AVAILABLE);
    if (availableMachines.length === 0) {
        return { statusCode: HttpResponseCode.NOT_FOUND };
    }
    const machine = availableMachines[0];
    // ... [Update logic omitted] ...
}
```

E

- What does `.filter(...)` do? Check the documentation (or ask an AI).
- It looks clean but hides an efficiency flaw: it walks the *entire* array. A location with 1,000 machines checks all of them, every time.
- `.find(...)` stops at the first match. Same $O(N)$ worst case, but it exits early whenever an available machine comes first.

Takeaways

- Once the logic works, check it briefly for efficiency.
- In an unfamiliar language or codebase, convenient-looking constructs are sometimes traps. Verify they do what you think.
- Ask (a search engine or an AI): is there a better way to do this?